Plans to port to dotnet natively

Status: plan, not implemented. There is no intermediate representation (IR), code generator, or assembly emitter in the repository today. The token interpreter and the image are the reference semantics. This document concludes the design discussion; it is not a decision record, and implementing any part of it needs a decision record and a commitments-ledger row.

This is a plan to compile Scale to .NET assemblies. It is not the existing NativeAOT publish path. NativeAOT publishes the F# interpreter (`docs/nativeaot.typ`); this plan emits IL from a lowered program and keeps the interpreter as the dynamic fallback.

The question

Two questions collapsed into one. Can Scale express a tree-walking type inferencer comfortably enough to justify a self-hosted bootstrap? And if the language is ever compiled to .NET assemblies, what is the smallest honest IR?

The first question is answered by the language, not by the emitter. The second is answered here: a small lowered discriminated union (DU) is the right seam, but the slogan is **classify everything, resolve the kernel, and guard the dynamic seams**, not *resolve everything*.

What the language already gives the compiler

Recursion is direct

The recursion objection does not hold. `define` stores a body and the evaluator splices that body back onto the program (`components/prelude/evaluator/evaluator.fs:296-303`), so a word can call itself by name. No fixed-point combinator is needed. Closures are first-class through `let`, `lambda`, and `apply` (`components/prelude/evaluator/closure.fs:27-83`), so higher-order traversal is available too.

A live-session spike, not a committed test, wrote a `countdown` word as `"countdown" { ... countdown ... } define` and printed `3 2 1 0`. Direct self-reference is already the idiom.

Heterogeneous trees are native

There are three interchangeable node representations, and the first is already the inferencer's substrate:

- **Terms.** `term` builds `head arg... arity term` compounds (`-2`); typed terms are `-3`. `unify` (with occurs-check and transactional failure), `resolve`, `lambda-term`, and `reduce` already walk them (`components/prelude/evaluator/environment.fs:581,647,664,677`). The engine an ML implementation would have to write is already a word here.
- **Classes, slots, and instances.** nominal nodes through `class`, `slot`, `create-instance`, `slot-value`, and `set-slot-value` (`components/prelude/evaluator/class-system.fs:212-410`).
- **Generic functions and methods.** class or value-set specializers select a method, and a method body can call the generic again (`components/prelude/evaluator/dispatch.fs:1298,1349-1439`). A spike defined an `infer` generic with `:leaf` and `:node` methods that recursed to `6`.

So the walker's case split is dispatch. It is not a missing `match`.

The IR verdict

Yes to a small lowered DU. No to the assumption that every word and every dispatch site is statically resolved.

The language is late-bound on purpose. `define` splices at call time, `control` and `word` can replace a form later, and a new generic method can be added at runtime. D19′s exact-specializer fast path is a cache, not compile-time resolution (`docs/kernel-inventory.typ`, risks). A compiler that freezes those seams in the name of "resolved words" changes the language.

The IR should therefore carry:

```
Instruction =
  | Literal of entry
  | PushSlot of slot
  | StoreSlot of slot
  | CallWord of wordId * guard option
  | Dispatch of genericId * sortTuple * guard option
  | Apply
  | Splice of codeRef
  | Fail of reason
  | RecordStep of token * resumableProgramCounter
```

A kernel word or a frozen user word lowers to a direct call. A late-bound word or generic lowers to `CallWord` or `Dispatch` with an optional cached target, a version or epoch guard, and a fallback to the runtime dispatcher. `Apply` is a first-class dynamic call. `Splice` preserves `define`/`control` semantics if the compiled artifact keeps redefinition.

This buys the two things the port actually needs: dispatch resolution and stack layout can be tested without emitting assemblies, and the System.Reflection.Metadata writer is a fold over opcodes with no language decisions left in it.

The emitter must be total

The SRM writer is mechanical only if every semantic decision already has an opcode. If it still matches on language constructs, or calls `executeWord`, the IR is not lowered enough. Metadata tokens, local allocation, branch targets, failure plumbing, and the type signatures of emitted methods are still work, but they are mechanical work. The test is not line count; it is whether the writer contains a language `match`.

History is part of the semantics

Today every executed token records a step before it runs, and the step records the token, the stack, the program to resume, and the program-counter metadata (`components/prelude/evaluator/evaluator.fs:355-389`). `rollback` restores the stack, program, and position; it does not capture the binding root (`components/prelude/evaluator/history.fs:134-169`). If a compiled artifact keeps `history`, bookmarks, and rollback, the IR needs a resumable program counter per step. If it drops them, that is an explicit scope decision, not an emitter detail.

Decision (D120). Compiled artifacts exclude history, bookmarks, and rollback. The lowered instruction set has no step record and no resumable program counter, `LoweredInstructions.historyScope` is `HistoryExcluded`, and the emitter writes that scope into the artifact metadata. The interpreter remains the only engine that records the event stream; the phase-2 tiering routes a program that uses `history`, `bookmark`, `jump`, or `rollback` to it. A test reflects over the closed instruction union and fails if a step-record instruction appears without this decision being revisited.

Stack slots are local, not global

Straight-line code and `if` lower to fixed slots. `apply` and multi-method dispatch do not have a statically known result effect: methods share the generic's argument arity, not necessarily its result arity. Either require a declared effect per closure and method, or model the operand stack at runtime

and treat slots as frame offsets inside straight-line regions. A single global slot assignment is not honest.

Case analysis without pattern matching

Dispatch is the primary case mechanism; no `match` construct is added. The decision follows the same no-duplicate-features rule that governs the rest of the language: the cell system and generic functions already own case selection.

What is lost is field extraction and closed-world exhaustiveness, not case dispatch. Two small recoveries are enough for the inferencer:

- **A binding form, not a case form.** `with-fields { left right } { ... }` binds slot values through `let` into a child frame, makes a closure of the body, and applies it so the caller frame is restored. It lowers to projections, pays back at every AST site, and does not reintroduce a closed case construct.
- **A coverage check through the dispatcher.** generate the table over the concrete leaf node classes and ask the dispatcher whether an applicable handler exists for each. A tag-to-handler map is not enough: a method on a superclass covers every subclass. Use nominal classes for AST nodes, not predicate-sort specializers, so the check is decidable. If the node universe can grow at runtime, re-run the check after a class or method declaration; if the artifact freezes it, one load-time check is enough.

This recovers the failure mode that matters, adding a node and forgetting a handler, on the first run rather than in production. It is a test-time check, not a compile-time proof.

It is also not the whole of exhaustiveness. A runtime table checks coverage; it does not prove that every method can be selected. For that, the existing decision-table machinery (D13, promoted as D73) already proves completeness and disjointness for a closed universe. The same shape could prove dispatch coverage over a declared closed node set. That is an optional proof obligation, and `Unknown` or `Unavailable` must be a failure, never a pass.

The four risks, classified

1. Field extraction and coverage: ergonomic and recoverable

This is the real per-site cost, and it is fixed by the binding form plus the dispatcher-backed coverage check. It is overhead, not a correctness gap, once the node universe is closed and nominal.

2. Dynamic typing: a finding, not necessarily a staging choice

The language can represent types as sorts, typed terms, and value sets, but it cannot check a definition's domain and codomain; generic dispatch is a runtime decision. A one-hour probe settles the question: write `infer` with a declared `Expr -> Type` signature and call it on a non-node. If the language accepts, the limitation is expressiveness, not schedule. Record it as a finding. If it is only slower to write statically, it is a reversible staging decision.

The construction side has a hole worth naming. Schema validation at node construction is a convention here because `create-instance` plus `set-slot-value` bypasses a smart constructor. Unless a construction guard is added, "validated node" is discipline, not a guarantee.

3. Substitution threading: bounded, but the bound is on transactional sites

The boilerplate is bounded by how many sites need transactional unification, not by the size of the compiler. Two caveats. First, keeping the old state is a snapshot only while the store is persistent; a

mutable compiled store needs an undo journal. Second, hiding `BindingTable` is not enough: every `Store.add*` returns a new store, so the pair `(Store, roots)` is what must leave the common path.

The language-level combinator also needs a primitive. `unify` is transactional internally, but branch-level backtracking needs a binding-root snapshot. `bookmark` and `rollback` are execution-history rollback, not a state snapshot (`components/prelude/evaluator/history.fs:134-169`). A narrow `checkpoint`/`restore` over the binding root is the bounded addition.

Whether the threading disappears is host-dependent. An F#-hosted bootstrap can hide it in a state monad or a mutable context. A Scale-hosted bootstrap makes it a language-feature question.

4. Non-compositional effects: a phase-3 problem

Phase 1 does not require the compiler to statically type its own `apply`. Quotations are opaque values, `apply` is a dynamic call, and effect typing is reserved for target-language programs. Full self-typing is a separate phase.

There is a codegen caveat. Opaque `apply` means the surrounding code cannot assign static stack slots across the call; phase 1 pays with a runtime stack or a spill/reload. Dynamic `apply` is an IR opcode, not a blocker. Static effect typing is an optimization and a self-typing question, not a correctness requirement.

Staging and order

The memory representation is frozen as of D90: managed backing, shared immutable values with promotion, no copy-on-send, and no evaluator routing through the arena. Phase 1 is therefore unblocked. The order follows from the cost classification: recover the cheap per-site costs first, keep the expensive semantic questions out of phase 1, and only then commit to an emitter.

1. **Phase 0 — record the plan.** This document. No code change, no new word, no new test. A real port starts with a decision record and a ledger row.
2. **Phase 1 — the executable IR.** Lower the kernel subset to the DU, run it on a small VM, and differentially test it against the token interpreter on the existing example corpus. Add `with-fields`. Add the closed nominal node universe and the dispatcher-backed coverage check. No assembly emission.
3. **Phase 2 — the SRM writer.** Only after the IR is total. Emit one method per code object, call the runtime library for store, dispatch, unification, and scheduler operations, and keep the interpreter as the fallback for live redefinition and for code the compiler declines.
4. **Phase 3 — self-typing.** Static effect and type inference for Scale's own quotations, if it is still wanted. This is where 4 is solved, not in phase 1.

The phase-2 tiering is the point of the guarded call sites: compiled code for closed words, interpreter fallback for live redefinition, and a version check at the seam. The lowered abstraction in phase 1 is also where the history/rollback decision is tested, before any metadata is written.

Acceptance

- The IR abstraction and the token interpreter agree on the example suite, including failures and diagnostics.
- The coverage check fails when a node class has no applicable handler and passes when a superclass handler covers it.
- A guard-invalidated call site falls back instead of running stale code.
- The SRM writer contains no language `match`; it matches only IR opcodes.

- The history decision is explicit: either resumable program counters are in the IR, or the compiled artifact states that history, bookmarks, and rollback are out of scope.
- A compiled artifact passes the same unfiltered suite as the interpreter for the compiled subset, and the interpreter still runs everything else.

Non-goals

- No `match` construct. The case mechanism is dispatch; the binding form is not a second case mechanism.
- No static typing of the compiler in phase 1.
- No full self-typing or effect system before a working self-hosted compiler.
- No assembly emission before the IR abstraction and the differential suite.
- No claim that the port is free of the image. The image remains the authority for anything that crosses an actor boundary, survives persistence, or is rolled back.

Open decisions

- Does the compiled artifact keep token-level history, bookmarks, and rollback?
- What is the closed-world boundary for the AST node universe?
- Persistent store, or mutable store plus an undo journal?
- What is the shape of the `checkpoint`/`restore` primitive?
- Does emitted code call the existing runtime library, or reimplement the operations in IL? Calling the library is smaller and preserves persistence; reimplementing is faster only after a profile says so.
- What is the version/epoch protocol for guarded word and dispatch sites?

Conclusion

The tree-walking inferencer is not the thing that sinks the effort. Recursion is direct, heterogeneous nodes are native, and the unifier is already a word. The sink risk is the non-compositional effect story around `apply` and `define`-splicing, and the right response is staging, not an effect system designed before the first working compiler.

For the native port, the right amount of abstraction is a small lowered DU: resolved where the kernel is frozen, classified and guarded where the language is late-bound, with history and stack layout represented rather than assumed. Build the IR and its VM first, prove it against the interpreter, then let the System.Reflection.Metadata writer be the mechanical fold it should be.

References

- `docs/decision-records.typ` — D4 (execution-state roots), D15 (unification), D19 (multiple dispatch), D29 (event stream), D13/D73 (decision tables).
- `docs/kernel-inventory.typ` — the bootstrap kernel and the cache-resolved words/methods risk.
- `docs/decision-models.typ` — Z3-proved completeness and disjointness.
- `docs/nativeaot.typ` — the existing NativeAOT publish path, which this plan does not replace.
- `docs/memory-and-go-transpiler.typ` — the related memory-tier and restricted transpiler analysis.
- `components/prelude/evaluator/evaluator.fs`, `components/prelude/evaluator/closure.fs`, `components/prelude/evaluator/environment.fs`, `components/prelude/evaluator/`

`dispatch.fs`, `components/prelude/evaluator/class-system.fs`, `components/prelude/evaluator/history.fs` — the semantics the IR must preserve.